

AIOS · DATA FLOW STRUCTURE

How Data Moves Through AIOS

Request → queue → engines → Google Sheets.

Every path a piece of data takes — from a client typing a URL, through the queue and the four engines, out to external APIs, and into Google Sheets as the operational store. Includes the JSON-to-Sheets storage change and the Postgres/Sheets split.

Inputs

URL-only, no user data

Engines

Audit · Content · Off-page · Radar

Store

Postgres + Google Sheets

Reporting

Live client workbooks

PREPARED FOR

Danyal
Agency Owner

PREPARED BY

Xegents

DATE

9 July 2026

THIS DOCUMENT

Data-flow
structure

The data model in one page

AIOS keeps two kinds of data apart on purpose. **Postgres** holds identity, relationships and secrets. **Google Sheets** holds the operational, client-facing records that used to be JSON files — now live, shareable and editable. Redis sits between the app and the slow work, buffering both API spend and Sheet writes.

2

STORES — POSTGRES + GOOGLE SHEETS

4

ENGINES PRODUCING DATA

0

USER LOGINS NEEDED TO AUDIT

JSON → GS

STORAGE REPLACED BY SHEETS

THE FIVE HOPS EVERY JOB MAKES

1 • Request

portal / API

2 • Enqueue

Redis + budget cap

3 • Engine runs

worker + vault keys

4 • Persist

Sheets (+ PG refs)

5 • Surface

portal + notify

WHERE EACH STORE IS USED

POSTGRES · PG

Auth & users, clients & sites, the **encrypted key vault**, activity log, and the Policy-Radar knowledge base. Secrets **never** leave here.

Google Sheets · GS

Audits, backlinks, citations, Web 2.0 properties, content jobs and milestones — one workbook per client, written by the workers.

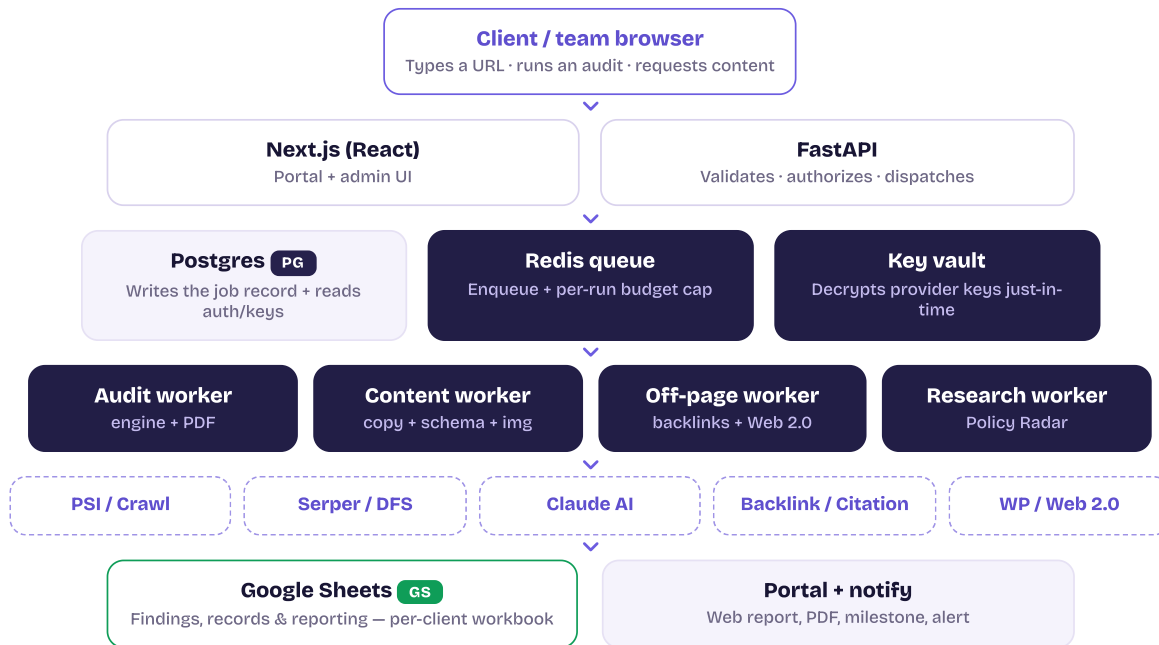
Redis

Job queue, concurrency caps, cached API responses, and the **write-buffer** that batches Sheet updates within quota.

02 THE MAIN PATH

Request to result

A single vertical path from browser to store. The request lane is fast and synchronous; everything expensive is pushed onto the queue so the UI never waits and spend stays capped.



Read the arrows as data, not calls: the browser→API hop carries the request; the worker→external hop carries API payloads; the worker→Sheets hop carries the structured result. Redis caches external responses so a repeat within TTL never re-bills.

03 STORAGE MODEL

Postgres vs Google Sheets

The storage change in one rule: **Google Sheets replaces the old JSON files** for operational and client-facing data; Postgres keeps only what a spreadsheet must never hold — identity, relationships and secrets. A thin **SheetStore** adapter gives the engines the same read/write interface they had for JSON.

ENTITY	STORE	KEY FIELDS	WRITTEN BY
users / sessions	PG	role, email, password_hash	Auth
clients / sites	PG	domain, cms_type, enc. WP creds	Admin
api_keys (vault)	PG	encrypted provider keys — never in a Sheet	Admin
activity_log	PG	actor, action, target, ts	All
KB: sources / entries / events / recs	PG	severity, category, region, source	Research
audits	GS	type, tier, run_uuid, scores, findings	Audit
content_jobs	GS	content_type, framework_used , target, cost	Content
backlinks	GS	ref_domain, anchor, authority, spam	Off-page
citations	GS	directory, nap_status, state	Off-page
web2_properties	GS	platform, post_url, anchor, verified	Off-page
milestones	GS	title, status, auto_source	All

WHY THE SPLIT

A Sheet is perfect for data the agency and client want to **see, share and edit live** — and useless (unsafe) for passwords and joins. So secrets and relationships stay relational; results become spreadsheets.

The write path (quota-safe)

Workers don't hit the Sheets API row-by-row. They push results into a **Redis buffer**, then a flusher writes them in batched batchUpdate calls — staying inside Google's per-minute quota even during a bulk crawl.

Audit & Content data flows

AUDIT — URL IN, FINDINGS + PDF OUT (NO USER DATA)



Optional owner-authorized enrichers (GSC / GA4) attach first-party traffic *if* connected — the URL-only flow above is complete without them.

CONTENT — BRIEF IN, PUBLISHED PAGE OUT (FRAMEWORK AUTO-SELECTED)

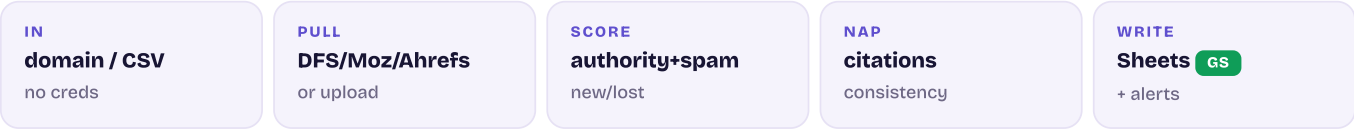


What the data carries

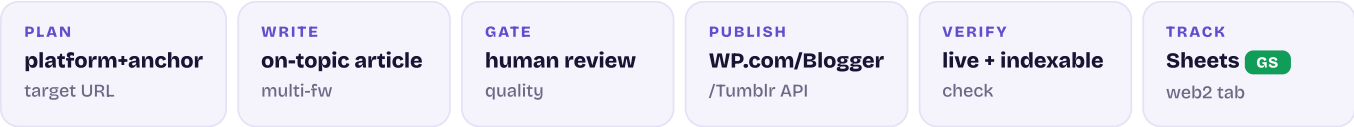
The content job record stores the **framework chosen** and the **search intent** that drove the choice, plus the published URL and per-job cost estimate — so the agency can see, per page, which structure was used and what it cost.

Off-page & Policy Radar data flows

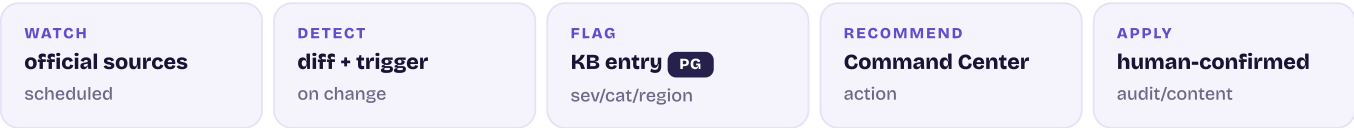
BACKLINKS & CITATIONS — DOMAIN IN, MONITORED PROFILE OUT



WEB 2.0 BACKLINK PIPELINE — ANCHOR IN, VERIFIED LIVE LINK OUT



POLICY RADAR — GOOGLE CHANGE IN, RECOMMENDATION OUT



Radar output stays in Postgres (versioned, source-cited) because it feeds live audit checks and needs joins — only the client-facing *advisory* is later mirrored to a Sheet.

06 CROSS-CUTTING

Lifecycle, cost control & security

HOW A VALUE FLOWS ACROSS THE BOUNDARIES

Ingested step 1-2

A URL or brief enters via the portal; FastAPI validates it and writes a

job record to Postgres, then enqueues it in Redis under the client's budget cap.

Processed step 3

A worker decrypts only the keys it needs from the vault, calls external APIs, and

caches each response in Redis so a repeat within TTL costs nothing.

Persisted step 4

Structured results buffer in Redis and flush **Google Sheet** to the client's

in batched writes; PDFs and artifacts go to file storage with a reference row.

Surfaced step 5

The portal reads the Sheet + PG references to render the web report, advances the milestone, and notifies the client — the agency can open the same live Sheet.

COST CONTROL IS IN THE DATA PATH

Budget caps live on the queue, caching lives in Redis, and tier gating decides **which paid API a job may even call**. A daily spend-stop halts paid providers if a run misbehaves — so data volume can never become a surprise bill.

Security invariants

- **Secrets never enter a Sheet** — keys stay in the encrypted vault.
- Secrets are never logged; keys are decrypted just-in-time per call.
- Audited-site & client content is treated as **data, not instructions**.
- Per-client workbooks scope what a client can ever be shown.

THE ONE LINE TO REMEMBER

Identity & secrets live in Postgres; everything the client sees lives in a Google Sheet — the JSON files are gone.

Xegents